Récursivité - exercices

Term NSI

1 Récursivité — fiche d'exercices

Terminale NSI — Durée indicative : 1 h 30 (hors prolongements)

Prérequis : cours sur la récursivité (partie 1) et TP.

Compétences travaillées - écrire une fonction récursive à partir d'une relation de récurrence ; - suivre l'exécution d'une fonction récursive (tableau de suivi, pile d'appels) ; - justifier la **terminaison** d'une fonction récursive (variant, préconditions) ; - compléter ou corriger une fonction récursive sur des chaînes et des tableaux, comme dans les sujets de bac.

---

2 Exercice 1 — Des suites aux fonctions récursives

2.1 Trois suites

a. $u_n = 2 \times u_{n-1} + 1$, avec $u_0 = 0$

b. $v_n = \frac{1}{v_{n-1}}$, avec $v_0 = 2$

c. $w_n = w_{n-1} + \frac{1}{w_{n-1}}$, avec $w_0 = 1$

1. Calculer à la main les 4 premiers termes de chaque suite.
2. Écrire les fonctions récursives `u(n)`, `v(n)` et `w(n)` qui renvoient le terme de rang `n`. Vérifier avec les valeurs de la question 1.
3. Quelle est la précondition sur `n` ? Que se passe-t-il si on appelle `u(-1)` ? Ajouter à la fonction `u` une instruction `assert` qui protège la fonction contre cette entrée.
4. Écrire une version **itérative** de `u(n)` (comme au TD).

2.2 Les vaches de Narayana

Une vache donne naissance à une autre vache tous les ans, en début d'année, à partir de sa quatrième année. Chaque nouvelle vache suit la même règle. On part d'une vache qui vient de naître ($v_1 = 1$). Le nombre de vaches les années suivantes est :

$$1, 1, 1, 2, 3, 4, 6, \ldots$$

1. Donner les 3 termes suivants de cette suite.
2. Le nombre de vaches de l'année `i` est celui de l'année précédente, auquel s'ajoutent les naissances. Expliquer pourquoi le nombre de naissances de l'année `i` vaut v_{i-3}. En déduire la relation de récurrence.
3. Écrire une fonction récursive `vaches(i)` qui renvoie v_i pour `i ≥ 1`.
4. Combien votre fonction a-t-elle de cas de base ? Un élève propose de n'écrire que `if i == 1: return 1`. Que se passe-t-il lors de l'appel `vaches(4)` ?

---

3 Exercice 2 — Fonction mystère

```
def f(x, y):
    if x == y:
        return x
    elif x < y:
        return f(x, y - x)
    else:
        return f(x - y, y)
```

1. Compléter le tableau de suivi pour l'appel `f(5, 15)`. Quelle valeur est renvoyée ?

appel avec x =	y =	return
5	15	f(5, 10)
5	10	

2. Même question pour `f(8, 29)`.

appel avec x =	y =	return
8	29	f(8, 21)
8	21	

3. Que calcule `f(x, y)` en général ?
4. **Terminaison.** On suppose que `x` et `y` sont des entiers strictement positifs. Montrer que `x + y` est un variant. En déduire que la fonction termine.
5. Que se passe-t-il pour l'appel `f(0, 5)` ? Quelle partie de la preuve n'est plus valable ? Écrire la précondition sous forme d'`assert`.

---

4 Exercice 3 — Palindromes

Un **palindrome** est un mot qui se lit de la même façon dans les deux sens, comme `"kayak"` ou `"radar"`.

4.1 Avec des tranches

```
1  def est_palindrome(s):
2      """renvoie True si la chaîne s est un palindrome, False sinon
3      >>> est_palindrome("kayak")
4      True
5      >>> est_palindrome("karma")
6      False
7      """
8      if len(s) <= 1:
9          return ...
10     if s[0] != ...:
11         return ...
12     return est_palindrome(...)
```

1. Compléter la fonction. On rappelle que `s[1:-1]` est la chaîne `s` privée de son premier et de son dernier caractère.
2. Représenter la pile d'appels pour `est_palindrome("radar")`, puis pour `est_palindrome("rotor")`... et `est_palindrome("route")`.
3. Justifier que la fonction termine. Quel est le variant ?

4.2 Avec des indices

Plutôt que de créer une nouvelle chaîne à chaque appel, on travaille sur la chaîne d'origine en ne regardant que la portion comprise entre les indices `i` et `j` (inclus).

```
1  def est_palindrome_ind(s, i, j):
2      """renvoie True si s[i], s[i+1], ..., s[j] forment un palindrome
3      >>> est_palindrome_ind("kayak", 0, 4)
```

```
    True
    """
    if i >= j:
        return True
    ...
```

1. Compléter la fonction.
2. Quel appel faut-il faire pour tester si une chaîne `mot` entière est un palindrome ?
3. Donner un variant de cette fonction.

---

5 Exercice 4 — Recherche dichotomique récursive

Soit `tab` un tableau non vide d'entiers triés dans l'ordre croissant et `x` un entier. La fonction `chercher` doit renvoyer un indice où la valeur `x` apparaît dans `tab` si cette valeur y figure, et `None` sinon.

Ses paramètres sont :

- `tab`, le tableau dans lequel s'effectue la recherche ;
- `x`, l'entier à chercher dans le tableau ;
- `i`, l'indice de début de la partie du tableau où s'effectue la recherche ;
- `j`, l'indice de fin de la partie du tableau où s'effectue la recherche.

```
def chercher(tab, x, i, j):
    '''Renvoie l'indice de x dans tab, si x est dans tab,
    None sinon.
    On suppose que tab est trié dans l'ordre croissant.'''
    if i > j:
        return None
    m = (i + j) // ...
    if ... < x:
        return chercher(tab, x, ... , ...)
    elif tab[m] > x:
        return chercher(tab, x, ... , ...)
    else:
        return ...
```

Exemples :

```
>>> chercher([1, 5, 6, 6, 9, 12], 9, 0, 5)
4
>>> chercher([1, 5, 6, 6, 9, 12], 6, 0, 5)
2
```

1. Recopier et compléter le code de la fonction.
2. Que renvoie `chercher([1, 5, 6, 6, 9, 12], 1, 0, 3)` ?
3. Compléter le tableau de suivi de l'appel `chercher([1, 5, 6, 6, 9, 12], 7, 0, 5)`. Que renvoie-t-il ?

appel avec i =	j =	m	return

4. **Terminaison.** On note `j - i + 1` la taille de la zone de recherche. Montrer qu'elle diminue strictement à chaque appel récursif. Pourquoi la fonction termine-t-elle toujours ?

---

6 Exercice 5 — Sujet bac Métropole, septembre 2021 (exercice 4)

Cet exercice porte sur la programmation en général et la récursivité en particulier. On s'intéresse à un algorithme de mélange des éléments d'une liste.

1. Pour la suite, il sera utile de disposer d'une fonction `echange` qui permet d'échanger dans une liste `lst` les éléments d'indice `i1` et `i2`. **Expliquer pourquoi le code Python ci-dessous ne réalise pas cet échange et en proposer une modification.**

```
def echange(lst, i1, i2):
    lst[i2] = lst[i1]
    lst[i1] = lst[i2]
```

2. La documentation du module `random` indique que `randint(a, b)` renvoie un entier aléatoire `N` tel que `a <= N <= b`. **Parmi les valeurs ci-dessous, lesquelles peuvent être renvoyées par l'appel `randint(0, 10)` ?**

```
0   1   3.5   9   10   11
```

3. Le mélange de Fisher-Yates permet de permuter aléatoirement les éléments d'une liste. En voici une mise en œuvre récursive :

```
from random import randint

def melange(lst, ind):
    print(lst)
    if ind > 0:
        j = randint(0, ind)
        echange(lst, ind, j)
        melange(lst, ind - 1)
```

a. Expliquer pourquoi la fonction `melange` se termine toujours.

b. Lors de l'appel de la fonction `melange`, la valeur du paramètre `ind` doit être égale au plus grand indice possible de la liste `lst`. Pour une liste de longueur `n`, **quel est le nombre d'appels récursifs de la fonction `melange` effectués, sans compter l'appel initial ?**

c. On considère le script ci-dessous :

```
lst = [v for v in range(5)]
melange(lst, 4)
```

On suppose que les valeurs successivement renvoyées par la fonction `randint` sont 2, 1, 2 et 0. Les deux premiers affichages produits par l'instruction `print(lst)` sont :

```
[0, 1, 2, 3, 4]
[0, 1, 4, 3, 2]
```

Donner les affichages suivants produits par la fonction `melange`.

d. Proposer une version itérative du mélange de Fisher-Yates.

Rappel : `range(debut, fin, pas)` parcourt les entiers de `debut` (inclus) à `fin` (exclu) avec le pas indiqué, qui peut être négatif. Par exemple `for i in range(4, 1, -1): print(i)` affiche 4, 3, puis 2.

7 Prolongements

7.1 P1 — La fonction 91 de McCarthy : un appel récursif dans un appel récursif

John McCarthy a défini cette fonction pour étudier la vérification formelle des programmes récursifs. Pour tout entier naturel `n` :

$$f(n) = \begin{cases} n - 10 & \text{si } \ n > 100 \\ f(f(n+11)) & \text{sinon} \end{cases}$$

1. Écrire cette fonction en Python.
2. Dérouler le calcul de `f(99)` :

```
f(99) = f(f(110))
      = f(100)
      = f(...
```

3. Calculer `f(n)` pour quelques valeurs de `n ≤ 100`. Que constate-t-on ?
4. Pourquoi est-il difficile de trouver un variant pour cette fonction ?

7.2 P2 — Un appel ou deux appels ? (lien avec la partie 2 : complexité)

On reprend la suite w de l'exercice 1.1. Deux élèves proposent :

```python
def w1(n):
    if n == 0:
        return 1
    return w1(n - 1) + 1 / w1(n - 1)

def w2(n):
    if n == 0:
        return 1
    x = w2(n - 1)
    return x + 1 / x
```

1. Les deux fonctions renvoient-elles le même résultat ?
2. On note A(n) le nombre total d'appels provoqués par `w1(n)` (appel initial compris). Établir une relation entre A(n) et A(n − 1). Même question pour `w2`.
3. Calculer A(10) dans les deux cas. Laquelle des deux fonctions faut-il préférer ?
4. Même question pour la fonction `vaches(i)` de l'exercice 1.2 : expliquer pourquoi elle devient très lente pour `i = 40`.

7.3 P3 — Exponentiation rapide

1. Écrire une fonction récursive `puissance(x, n)` qui calcule x^n en utilisant $x^n = x \times x^{n-1}$. Combien de multiplications fait-elle ?
2. On remarque que $x^n = (x^{n/2})^2$ si `n` est pair, et $x^n = x \times (x^{(n-1)/2})^2$ si `n` est impair. Écrire une fonction récursive `puissance_rapide(x, n)` utilisant ces relations (attention à ne faire qu'**un seul** appel récursif).
3. Compter les multiplications effectuées pour calculer x^{16} avec chacune des deux fonctions.

7.4 P4 — Dessin récursif : le tapis de Sierpiński

On considère un carré de côté 1. On le partage en 9 carrés égaux et on colorie le carré central. Puis, pour chaque carré non colorié, on réitère le procédé. On note u_n l'aire coloriée après l'étape `n`, avec $u_0 = 0$:

$$u_{n+1} = \frac{8u_n}{9} + \frac{1}{9}$$

1. Représenter le dessin obtenu à l'étape 2.
2. Écrire la fonction récursive `aire(n)` qui renvoie u_n. Déterminer expérimentalement la limite de la suite.
3. ⋆ Avec le module `turtle`, écrire une fonction récursive `tapis(x, y, cote, n)` qui dessine le tapis à l'étape `n`.